Database Enhancement

Student ID: 12210505

Donghang Liu

GitHub Repository

Shenzhen, Guangdong

December 2025

Contents

Chapter 1

Database Enhancement

1.1 Introduction

Here, we aim to enhance the original database. Given that the data source of the original database is unknown, it is extremely challenging to accurately locate the exact corresponding movie for each entry in the movie table.

Furthermore, subsequent maintenance of the database will also be cumbersome. To address these issues, we will leverage IMDb—the world ' s largest movie database—to facilitate precise movie localization. IMDb assigns a unique primary key to each movie, allowing full access to a movie ' s details via the URL: https://www.imdb.com/title/ + [the primary key provided by IMDb]. Additionally, IMDb offers non-commercial datasets on its official developer platform (available at: https://developer.imdb.com/non-commercial-datasets/). This resource not only enables us to perform data enhancement effectively but also simplifies future database expansion for other developers. For the latest films, we will randomly select approximately 150 films released in 2017, and around 300 films annually for each year from 2018 to 2025 to supplement the database.

1.2 Methodology

The original dataset exhibits several limitations:

- Inconsistent film titles arising from variations in translations
- Minor discrepancies between the release years/runtimes in the dataset and the corresponding records in IMDb
- different release version of the same movie causing the inconsistency of data

Additionally, several critical issues require addressing:

- How to accurately map the existing database records to the IMDb non-commercial dataset?
- How to structure the storage of films produced through international co-productions (involving multiple countries)?
- What measures can be implemented to facilitate future database enhancements following the initial improvement?

Generally speaking, we follow three steps for data matching, integration, and enhancement:

Step 1: Match the movies in the existing database against those in the IMDb database, and record the corresponding IMDb primary key for each movie.

Step 2: Randomly select IMDb movie data from 2017 to 2025, and log their IMDb primary keys.

Step 3: Crawl movie data from the IMDb database using the primary keys identified in the previous two steps, then import the crawled data into our database.

1.3 Data Preprocessing

1.3.1 IMDb Introduction

IMDb (the International Movie Database) is the world's most popular and authoritative source for movie, TV and celebrity content. It offers Non-Commercial movie dataset, which is crucial in database enhancement process.

1.3.2 IMDb Non-Commercial Datasets

`https://datasets.imdbws.com/` is the subsets of IMDb data are available for access to customers for personal and non-commercial use, the data is refreshed daily.

In this datasets, we mainly focus on two tables: name_basics and table_basics. Among these two tables, I noticed that there are **tconst** and **nconst**, acted as **unique identifier** for movies and persons respectively!

name_basics:

- nconst (string) - alphanumeric unique identifier of the name/person
- primaryName (string)–name by which the person is most often credited
- birthYear –in YYYY format
- deathYear –in YYYY format if applicable, else '\N'
- primaryProfession (array of strings)–the top-3 professions of the person
- knownForTitles (array of tconsts) –titles the person is known for

table_basics:

- tconst (string) - alphanumeric unique identifier of the title
- titleType (string) –the type/format of the title (e.g. movie, short, tvseries, tvepisode, video, etc)
- primaryTitle (string) –the more popular title / the title used by the filmmakers on promotional materials at the point of release
- originalTitle (string) - original title, in the original language

- isAdult (boolean) - 0: non-adult title; 1: adult title
- startYear (YYYY) –represents the release year of a title. In the case of TV Series, it is the series start year
- endYear (YYYY) –TV Series end year. '\N' for all other title types
- runtimeMinutes –primary runtime of the title, in minutes
- genres (string array) –includes up to three genres associated with the title

1.3.3 Import Database

```
DROP TABLE IF EXISTS name_basics;
CREATE TABLE name_basics (
    nconst VARCHAR(20) PRIMARY KEY,
    primaryName VARCHAR(255) NULL,
    birthYear INT,
    deathYear INT,
    primaryProfession VARCHAR(255),
    knownForTitles VARCHAR(255),
    gender CHAR(1)
);
COPY name_basics (nconst, primaryName, birthYear, deathYear,
   primaryProfession, knownForTitles) FROM 'D:\\Database␣Project2\\
   name.basics.tsv' WITH (
    FORMAT text,
    DELIMITER E'\t',
    HEADER,
    NULL '\N'
);
DROP TABLE IF EXISTS table_basics;
CREATE TABLE table_basics (
    tconst VARCHAR(20) PRIMARY KEY,
    titleType VARCHAR(25),
    primaryTitle VARCHAR(511),
    originalTitle VARCHAR(511),
    isAdult INT,
    startYear INT,
    endYear INT,
    runtimeMinutes INT,
    genres VARCHAR(255)
);
```

```
29
30 COPY table_basics (tconst, titleType, primaryTitle, originalTitle,
      isAdult, startYear, endYear, runtimeMinutes, genres) FROM 'D:\\
      Database␣Project2\\title.basics.tsv' WITH (
31     FORMAT text,
32     DELIMITER E'\t',
33     HEADER,
34     NULL '\N',
35     ENCODING 'UTF8'
36 );
```

We also need to the insert the index for title, runtime, release year to enable the query speed.

```
1 CREATE INDEX idx_movies_year_movieid ON movies (year_released,
      movieid);
2
3 CREATE INDEX idx_table_basics_startyear ON table_basics (startyear)
      ;
4
5 CREATE INDEX idx_table_basics_runtimeminutes ON table_basics (
      runtimeminutes);
6
7 CREATE INDEX idx_movies_title_trgm ON movies USING GIN (title
      gin_trgm_ops);
8
9 CREATE INDEX idx_table_basics_originaltitle_trgm ON table_basics
      USING GIN (originaltitle gin_trgm_ops);
10 CREATE INDEX idx_table_basics_primarytitle_trgm ON table_basics
      USING GIN (primarytitle gin_trgm_ops);
```

1.4 Movie Matching

Here I will present the full process of movie matching, and here we use the 3 main identifiers to do so: **title,release year and runtime**. Consider that these data of the movie might be mistakenly recorded, we might have several steps, each step has an different tolerance factor for these 3 identifiers.

1.4.1 Step 1: Rough Match

While pinpointing a movie' s exact day of release can be challenging, there is no justification for significant discrepancies in its release year—at most a one-year difference may occur (e.g., Chinese films released during Spring Festival). Due to translation variations, some movies may lack an exact name match but instead have semantically similar titles. Runtime data is also prone to inconsistencies,

as films are often edited for release in different countries to comply with regional legal requirements.

Accordingly, we implemented a matching process using the similarity function in the pg_trgm extension, adhering to the following rules:

- The movie must have the same release year as its counterpart in the IMDb database.
- The movie’ s title must be similar to the corresponding title (original title or primary title) in IMDb.
- If multiple entries satisfy the first two criteria, the one with the smallest runtime difference shall be selected.

Then here’s the corresponding code

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
DROP TABLE IF EXISTS movie_matched_001;
CREATE TABLE movie_matched_001 AS
SELECT
  ranked.*
FROM (
  SELECT
    m.movieid,
    t.tconst,
    m.title,
    t.originaltitle,
    t.primarytitle,
    m.runtime,
    t.runtimeminutes,
    m.year_released,
    -- calculate max similiarty
    GREATEST(
      similarity(m.title, t.originaltitle),
      similarity(m.title, t.primarytitle)
    ) AS max_similarity,
    -- calculate the absolute value for the difference of the
       runtime
    ABS(m.runtime - t.runtimeminutes) AS runtime_diff,
    --windows function
    ROW_NUMBER() OVER (
      PARTITION BY m.movieid
      ORDER BY
        GREATEST(similarity(m.title, t.originaltitle), similarity(m
           .title, t.primarytitle)) DESC,
```

```
        ABS(m.runtime - t.runtimeminutes) ASC
    ) AS rn
  FROM movies m
  INNER JOIN table_basics t
    ON t.startyear = m.year_released
  WHERE
    (similarity(m.title, t.originaltitle) >= 0.5 OR similarity(m.
       title, t.primarytitle) >= 0.5)
) AS ranked
WHERE ranked.rn = 1 --the most matched only
ORDER BY ranked.movieid;
```

Upon creating the table, we conducted a validity check to ensure its accuracy. Specifically, we verified that movies with similar titles correspond to the same film. We then queried the movie_matched_001 table (created in the preceding step) to **identify matched movies that meet all of the following criteria**:

1. The absolute runtime difference exceeds 5 minutes;
2. The titles do not match exactly.

During this verification process, we discovered that the runtime of some movies had been erroneously recorded as ”0” in the original database. Finally, we exported the relevant records to CSV files for ease of access and further analysis, with the final implementation code provided below:

```
COPY
(select movie_matched_001.*
from movie_matched_001
join movies m on m.movieid = movie_matched_001.movieid
where runtime_diff > 5 and max_similarity!=1 and m.runtime != 0
    )
to 'D:\\Database␣Project2\\unmatched_001.csv' with (FORMAT csv,
   HEADER, ENCODING 'UTF8');
```

There’s only 91 rows and we check it out manully.

1.4.2 Step 2:Further Matching

During the manual check, I noticed that the release dates of some films were recorded incorrectly (either one year early or late). Additionally, some film entries, though not present in the existing IMDb dataset, can still be found on the official IMDb website. Therefore, our second step will involve matching the 798 previously unmatched records.

Given the small dataset size, we can increase the tolerance for fuzzy matching here. Ultimately, we decided to perform a fuzzy match for each film against all entries within a one-year window (±1

year of its recorded release date). We will select the entry with the closest title match. If multiple entries share an identical title, we will prioritize the one with the smallest difference in runtime.

```
DROP TABLE IF EXISTS movie_matched_002;
CREATE TABLE movie_matched_002 AS
SELECT ranked.*
FROM (
  SELECT
    m.movieid,
    t.tconst,
    m.title,
    t.originaltitle,
    t.primarytitle,
    m.runtime,
    t.runtimeminutes,
    m.year_released,
    t.startyear AS matched_year,
    s.max_sim AS max_similarity,
    ABS(m.runtime - COALESCE(t.runtimeminutes, 0)) AS runtime_diff,
    ROW_NUMBER() OVER (
      PARTITION BY m.movieid
      ORDER BY s.max_sim DESC, ABS(m.runtime - COALESCE(t.
         runtimeminutes, 0)) ASC
    ) AS rn
  FROM movie_unmatched_001 m
  CROSS JOIN LATERAL (
    SELECT
      t.tconst,
      t.originaltitle,
      t.primarytitle,
      t.runtimeminutes,
      t.startyear,
      GREATEST(
        similarity(m.title, t.originaltitle),
        similarity(m.title, t.primarytitle)
      ) AS max_sim
    FROM table_basics t
    WHERE t.startyear BETWEEN (m.year_released - 1) AND (m.
       year_released + 1)
      AND (
        similarity(m.title, t.originaltitle) >= 0.5
```

```
        OR similarity(m.title, t.primarytitle) >= 0.5
      )
  ) t,
  LATERAL (SELECT t.max_sim AS max_sim) s
) AS ranked
WHERE ranked.rn = 1
ORDER BY ranked.movieid;
```

As a result, we found 228/798 records. Then we do the same consistency checking process. Finally, find the movies that we still didn't match (570 left).

```
CREATE TABLE movie_unmatched_002 AS
(SELECT m.*
FROM movie_matched_001 m
LEFT JOIN movie_matched_002 mm002
  ON m.movieid = mm002.movieid
WHERE mm002.movieid IS NULL );
```

1.4.3 Step 3: Final Search

Here we need to search for those movies that still couldn't be matched. For some reasons, I couldn't explain the steps very clearly. All you need to know is that I search it on IMDb official website and get the corresponding data(IMDb primary key). Then generate it into table: "table_basics_update"

```
DROP TABLE IF EXISTS movie_matched_003;
CREATE TABLE movie_matched_003 AS(
select tt.* from movie_unmatched_002 m
inner join table_basics_update tt
on tt.movie_id = m.movieid
where  abs(tt.year_released-m.year_released)<=1);


DROP TABLE IF EXISTS movie_unmatched_003;
CREATE TABLE movie_unmatched_003 AS(
select tt.* from movie_unmatched_002 m
inner join table_basics_update tt
on tt.movie_id = m.movieid
where  abs(tt.year_released-m.year_released)>1);
```

And finally 103 remaining datas need to be validate. Then we finished the matching process.

1.5 Movie Data Crawling

Our next step is to crawl data from IMDB based on the previous matching results(and we also randomly choose some recent movies). Following the design rule of IMDB's movie webpage URL – "https://www.imdb.com/title/" + "the primary key of the movie on IMDB" –we will write a Python file to retrieve the movies' information. (Note that headers and cookies have been removed from the Python file.)

We retrieve the movie's information, along with the cast, and record it in nconst form(the primary key for people designed by IMDb). As we cannot ensure the data validity in the original database, here we directly use the IMDb primary key to identify each people.

1.6 Database Enhancement

Finally, we imported the crawled data into the database, which yielded two new tables: movies and credits. Based on the cast members recorded in the credits table, we only retained those who had participated in at least one film that exists in the database.

```
DELETE FROM name_basics
WHERE nconst IN (
    SELECT nb.nconst
    FROM name_basics nb
    LEFT JOIN merged_unique_data mud ON nb.nconst = mud.people_id
    WHERE mud.people_id IS NULL
);
```

As a result, we enhanced the database, ensuring 100% accuracy of the data and 99% consistency with the original dataset. Please refer to the SQL files for details.

1.7 Remark

You can get the url of people and movie in my database, just search:

1. **"https://www.imdb.com/title/"+"imdb_id" for movie's homepage.**
2. **"https://www.imdb.com/name/"+"peopleid" for people's homepage**

And you can download my final SQL file here, it's far from perfect of course, and we can't do the perfect job no matter how hard.

Chapter 2

Lecture Notes Review

2.0.1 Overview

I' m already a senior this year, so technically I don' t need to take this course. Even if I do take it, I just need a passing grade to avoid affecting my graduation. The only reason I took this course is that I want to learn some engineering knowledge. So I won' t waste energy nitpicking this section; instead, I' ll only use it to review the course content and organize the key knowledge points. Additionally, I' ll compile a summary of document errors related to the lab sessions based on the lab assignments I submitted earlier (which include some mistakes I already found).

2.1 Week 1

The first week focus on Database introduction and basic environment setup. For this phase, I highly recommend a CSDN blog that proved incredibly helpful to me: `https://blog.csdn.net/m0_74282695/article/details/134793443/`. This comprehensive guide walks through the complete process of installing, configuring, and launching a PostgreSQL database, with clear step-by-step instructions and practical examples.

2.2 Week 2

This week mainly focus on the basic operations for the database(insert,delete,update and creating tables). From now on, examples for gaussdb is required.

The first difference for GaussDB is that it can be compatible with MySQL and PostgreSQL without additional configuration

```
--gaussdb Database creating operation
    CREATE DATABASE database WITH DBCOMPATIBILITY='PG' --or WITH
       DBCOMPATIBILITY='MySQL'
```

GaussDB uses UTF8 by default and also supports Chinese encodings such as GBK. By contrast, PostgreSQL also defaults to UTF8 but requires manual configuration to enable GBK encoding. In this regard, GaussDB offers more user-friendly support for Chinese encodings and does not need additional compilation.

Finally, to ensure compatibility with MySQL, GaussDB converts pure whitespace into NULL

by default, whereas PostgreSQL strictly distinguishes between whitespace, empty strings, and NULL. So:

```
    -- gaussdb
    CREATE TABLE test (col VARCHAR(20));
    INSERT INTO test VALUES ('␣␣␣'); -- deposit NULL
    SELECT col IS NULL FROM test; -- return true

    -- method: close BLANK_AS_NULL
    SET BLANK_AS_NULL = off;
    INSERT INTO test VALUES ('␣␣␣'); -- deposit blank
    SELECT col IS NULL FROM test; -- return false
```

2.3 Week 3

The third week focus basic operations, basic syntax for retrieving data from one table, and some datatypes like time and a special data ”Null”.

2.3.1 Page 44

Unavoidably, we need to update gaussdb here:

```
--how to add one month using Gaussdb?
    date_col + interval '1␣month'
--if Gaussdb compatible with Oracle
    add_months(date_col, 1)
```

2.3.2 Null

Since GaussDB is compatible with databases like Oracle and MySQL, this means that when it works with different databases, it follows the corresponding syntax rules.

In Oracle compatibility mode, empty strings are evaluated as NULL.

```
gaussdb=# CREATE DATABASE db_test1 DBCOMPATIBILITY = 'ORA';
gaussdb=# \c db_test1
db_test1=# SELECT '' IS NULL;
 ?column?
----------
 t
(1 row)
```

In MySQL compatibility mode, empty strings are not evaluated as NULL.

```
gaussdb=# CREATE DATABASE db_test2 DBCOMPATIBILITY = 'MYSQL';
```

```
gaussdb=# \c db_test2
db_test2=# SELECT '' IS NULL;
 ?column?
----------
 f
(1 row)
```

2.3.3 Page 55

```
--gaussdb(default)
\d movies
```

However, gaussdb is compatible with PGSQL by default.

2.4 Week 5

2.4.1 different notations

left join and left outer join have no difference in practice in main database systems.

2.5 Week 6

2.5.1 Page 11

For gaussdb:

```
ORDER [SIBLINGS] BY { column_name | number | expression } [ ASC |
   DESC ][ NULLS FIRST | NULLS LAST ] [ , ... ]
```

null is greater than anything by default.

2.5.2 Page 22-26

Gaussdb actually compatible with them all.

2.6 Week 7

2.6.1 Fuzzy Search

pg_trgm is an extension used for string similarity matching in PostgreSQL (and GaussDB). Among its features, the **similarity** function is a core component that calculates the trigram similarity between two strings.

The underlying principle works as follows: pg_trgm splits each string into a set of consecutive

three-character segments (trigrams). It then computes the ratio of the intersection of the trigram sets from the two strings, ultimately deriving a similarity score.

In Chapter 1, this function was also used to verify the consistency of movie titles.

2.6.2 Sequence Function in Gaussdb

No need to remember just need to know there are such functions.

1. nextval(regclass)
2. currval(regclass)
3. lastval()
4. setval(regclass, bigint)

2.6.3 Gauss Database file loading

1. sql file

   \i in command line

2. .csv file, txt file, etc

   \copy or copy (no difference with postgreSQL)

2.7 Week 8

2.7.1 Page 49

Procedural extensions to Gaussdb are PL/pgSQL.

2.8 Week 9

2.9 Week 10

2.9.1 Page 88

In Gaussdb, the commands are the same with postgreSQL.

- EXPLAIN: Generates only the estimated execution plan (no SQL execution)
- EXPLAIN ANALYZE: Executes the SQL and shows the actual execution plan + runtime metrics

Hence, I used the latter one in the performance evaluation in Project 1.

2.9.2 Index

For the introduction of index, the in-depth knowledge is necessary to be performed(e.g. B Tree, B+ Tree), which seems extremely useful when creating the index.

2.10 Week 11

Introduce Schema and View and their relationship. No need to switch the lecture file as the in class explaination is clear enough.

2.11 Week 12

2.11.1 GaussDB Official Document

Create, modify, and delete schemas

1. To create a schema, see CREATE SCHEMA. By default, initial users and system administrators can create schemas, and other users need to have the CREATE permission of the database to create schemas in the database.

2. To change the name or owner of a schema, see ALTER SCHEMA. Schema owners can change the schema.

3. To delete a schema and its objects, see DROP SCHEMA. Schema owners can delete Schema.

4. To create a table in a schema, create the table in the schema_name.table_name format. If no schema_name is specified, the object is created in the first Schema in the search path by default.

5. To view the Schema owner, perform the following correlation query GS_USER the system table PG_NAMESPACE and system view. Replace the schema_name in the statement with the name of the schema you are actually looking for.

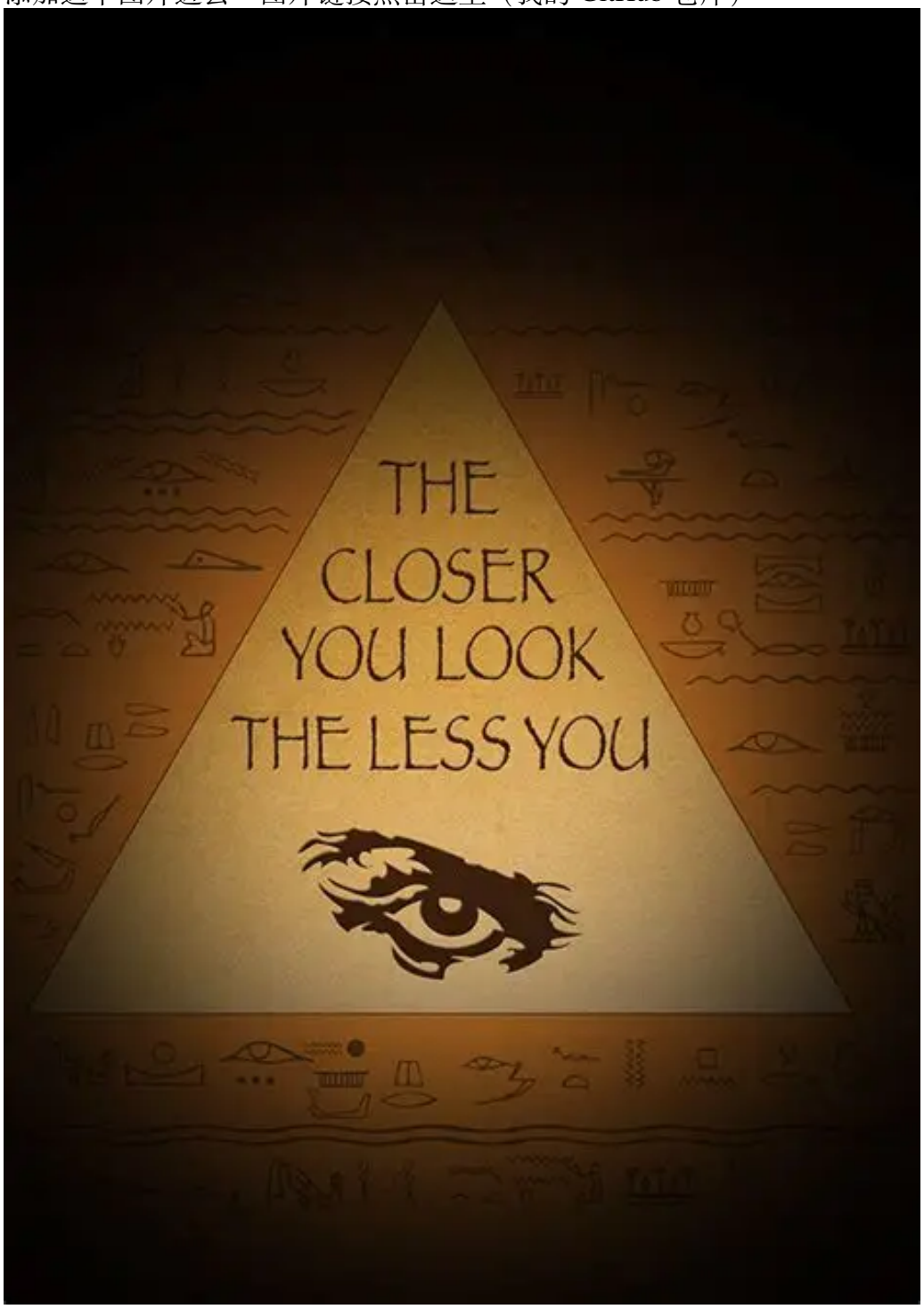
最后是有关课程最后提议的一些看法和想法。这里不属于 project 内容，我也没必要用英文写一遍再找 AI 去改语法错误。

首先这个观点是完全正确的。如今代码的上手门槛越来越低了，从原来的查找资料，手写程序，程序修改整套流程，到现如今询问 AI，AI 编程，超过 50% 概率直接就能跑。我本人是数学系的，上个学期期末，我用了两个下午就可以完全实现一个使用 pytorch 梯度下降法求解偏微分方程的代码。当然，在整个流程之前，我甚至不知道 pytorch 是什么。所以说，如果把同样的问题交给一个计系的同学，他可能比我编的代码更快，但是不会有质的差距。我作为理学院的学生，虽然不该但是可以理直气壮地把编程中的大部分交给 AI，自己去调整逻辑问题，一边调整一边学习，但是我不确定如果一个婴儿生下来就有拐杖用，他能不能在他应该跑步的年纪跑的比别人快。

但是，如果这个项目想去运转，那么光靠热情是不够的。人的初心是会变的。即是不变，人也是会更迭的。这个时候，一个组织能否恪守初心便成了一个随机数。一个组织的开创者如何能让组织经历了超过 2 代的更迭后，仍然记得这个组织开创的初心？这是一个值得去思考的问题。在我看来，理科和逻辑打交道，工科和现实打交道，但是如果想去涉足商科，这种和人打交道的事情，还是需要有商科的人的建议的，因为人是最没有理智，最没有逻辑的，也最不现实的。

我写不出“所以”，因为我也想不明白这个想法会最后以什么样子的形式实现，更别提提出一个一定可靠的建议了。也许这就像数据库，越想看的具体，看到的越模糊。**最后，一个关于第一节课的 ppt 的一个改进建议，也是本篇最有意思的改进建议：**

添加这个图片进去：图片链接点击这里（我的 GitHub 仓库）



2.12 Lab Material Correction

See Appendix.

Chapter 3

Course Content Proposal

3.1 Python Web Crawlers

3.1.1 Design Purpose

Upon the completion of Task 1, we have not only ensured the proper maintenance of the database itself, but also streamlined the maintenance workflow for subsequent administrators. In the solution developed for this task, we have associated each film with its unique **Primary Key** in the IMDB database. This design serves as a core enabler for precise data traceability and cross-platform data association.

However, the true value of a database lies in its **up-to-dateness and completeness—authoritative** platforms such as IMDB continuously update film-related data including ratings, box office figures, cast members, and award information. Relying solely on manual data entry or updates is not only inefficient, but also prone to errors and data staleness. Therefore, acquiring basic web crawling skills is essential, especially for students in the **Honor Class**.

3.1.2 Structure

Basic knowledge

Nature of Network Requests

Understand the **Request-Response Model** of the HTTP/HTTPS protocol: a crawler sends a request to the IMDB server, which then returns a response containing movie-related data. This constitutes the underlying logic of data collection.

Core Python Libraries and Tools

Focus on libraries directly relevant to movie data collection and clarify their core functions:

- **requests**: Sends HTTP requests and supports configuring request headers (e.g., User-Agent) to simulate browser access, thereby avoiding direct blocking by the IMDB server.

Data Format Recognition

Distinguish between two core data carriers:

- **HTML**: The data format used on IMDB web pages, which requires parsing to extract target information.

- **JSON**: The format returned by API interfaces, featuring a high degree of structuring.

Identify the presentation characteristics of movie data in different formats to lay the groundwork for subsequent parsing.

IMDB Data Characteristics

Learn the storage rules of IMDB movie data. For example:

- Each movie is assigned a unique primary key (**IMDB ID**, typically in the format of ttXXXXXXX).
- Core information (e.g., title, rating, director) is displayed in fixed sections on the page.

These rules serve as the basis for targeted data collection.

HTML analysis

Obtain HTML Source Code:

Use the requests library to send requests to the target page, and configure appropriate request headers (e.g., User-Agent: Mozilla/5.0) to simulate browser access, thereby avoiding being blocked by anti-crawling mechanisms.

Locate Core Data Areas:

Inspect the page structure using the browser’ s **Developer Tools (F12)** to identify the HTML tag blocks where core movie information resides (e.g.,<script type=”application/ld+json”>).

Extract Target Fields:

Write parsing code tailored to the fields required by the movie database.

Initial Data Cleaning:

Remove spaces and special characters from the fields (e.g., line breaks in titles, redundant decimal places in ratings), and standardize the data format to comply with the storage requirements of the database (e.g., convert ratings to float type and release years to int type).

Write a crawler

Core Requirements:

Batch collect IMDB movie data (including IMDB ID, title, director, cast, release year, rating, genre, synopsis, etc.) and synchronize it to the local movie database; support incremental updates based on IMDB ID (only collect movie data that is not stored in the database or has been updated).

Crawler Workflow:

Prepare a list of movies to be collected (existing IMDB IDs can be retrieved from the database, or batch obtained from IMDB movie category pages); Iteratively access the detail page of each movie, send requests and parse the data; Perform cleaning and format validation on the parsed data;

Crawler Optimization Techniques

Anti-Crawling Evasion:

Set random request intervals (e.g., 0.5-2 seconds), use a proxy IP pool (for scenarios where IMDB has strict anti-crawling measures), and rotate User-Agents; Efficiency Improvement: Adopt

multi-thread/multi-process collection (e.g., using the threading library) and batch insert/update data (integrate multiple pieces of data into a single SQL statement for execution);

Error Tolerance Handling:

Add a request retry mechanism (e.g., using try-except blocks to catch request timeout errors) to prevent crawler termination caused by exceptions in individual data entries.

Crawler Compliance: robots.txt

Essence of the robots.txt Protocol

The robots.txt file is a text document located in the root directory of a website server. It serves to inform crawlers of "which pages are crawlable and which are prohibited from being crawled", acting as a "gentlemen's agreement" between the website and crawlers. While it does not carry legal enforceability, violating this protocol may result in IP blocking and even potential legal risks.

Accessing IMDB's robots.txt

Access Method: Directly visit `https://www.imdb.com/robots.txt` to retrieve IMDB's crawler rules;

Additional Compliance and Ethical Requirements

Legal Boundaries:

Strictly comply with the *Cybersecurity Law* and *Personal Information Protection Law*. Collecting personal privacy information is prohibited; only publicly available movie data may be collected;

Traffic Control:

Avoid imposing excessive pressure on IMDB servers through high-frequency requests. It is recommended that the number of requests per IP does not exceed 1 per second;

Data Usage:

The collected movie data shall only be used for internal database maintenance, and shall not be used for commercial profit or illegal distribution.

3.1.3 Lab material

See appendix 1, or you can find it in my github repository.

3.1.4 References For Chapter 3

1. 韦世东. **Python 3 反爬虫原理与绕过实战** [M]. 北京: 人民邮电出版社, 2020. ISBN 978-7-115-52873-5.

2. (美) 瑞安 · 米切尔 (Ryan Mitchell). **Python 网络爬虫权威指南（第 2 版）**[M]. 神烦小宝译. 北京: 人民邮电出版社, 2019. ISBN 978-7-115-50926-0.

Appendix .1 Additional Materials

python crawler Tutorial

1.robots.txt

`robots.txt` is a text file placed in a website's root directory (e.g., `https://www.imdb.com/robots.txt`). It defines **rules for crawlers** (which pages to crawl/avoid) — a "gentlemen's agreement" between websites and crawlers (not legally binding, but violating it may lead to IP bans or legal risks).

Step 1: How to View a Website's `robots.txt`

To check IMDB's crawler rules, simply visit:

```
https://www.imdb.com/robots.txt
```

Step 2: Key Components of `robots.txt`

A typical `robots.txt` file uses two core directives:

- `User-agent`: Specifies which crawler the rule applies to (use `*` for all crawlers).
- `Disallow`: Lists pages/paths **prohibited** for crawling.
- `Allow`: Lists pages/paths **allowed** for crawling (overrides `Disallow` if conflicting).

Step 3: Example Analysis (IMDB's `robots.txt`)

IMDB's `robots.txt` includes rules like:

```
# robots.txt for https://www.imdb.com properties
# See https://m.imdb.com/robots.txt for the mDot equivalent

User-agent: *
Disallow: /OnThisDay
Disallow: /*/OnThisDay
Disallow: /ads/
Disallow: /*/ads/
Disallow: /ap/
Disallow: /*/ap/
Disallow: /mymovies/
Disallow: /*/mymovies/
Disallow: /register
Disallow: /*/register
Disallow: /registration/
Disallow: /*/registration/
......

User-agent: Baiduspider
Disallow: /list/*
Disallow: /*/list/*
Disallow: /user/*
Disallow: /*/user/*
```

```
User-agent: bingbot
Disallow: /showtimes/location/*
Disallow: /*/showtimes/location/*

User-agent: anthropic-ai
Disallow: /
User-agent: Claude-Web
Disallow: /
User-agent: CCbot
Disallow: /
User-agent: FacebookBot
Disallow: /
User-agent: Google-Extended
Disallow: /
User-agent: GPTBot
Disallow: /
User-agent: PiplBot
Disallow: /
```

interpretation:

- All crawlers (`User-agent: *`) **cannot** crawl `/ads/`, `/ap/`, or `/mymovies/`.
- BaiduSpider is **prohibited** from crawling:
    - Pages under `/list/` (e.g., `https://example.com/list/movies`)
    - Nested `/list/` paths (e.g., `https://example.com/category/list/top10`)
    - User-related pages (e.g., `https://example.com/user/profile`, `https://example.com/group/user/posts`)
- Crawlers like Anthropic's AI, GPTBot, Claude-web, CCBot, FacebookBot, Google-Extended, and PipIBot are **completely blocked**
- Crawlers **can** crawl `/title/` (movie detail pages, e.g., `https://www.imdb.com/title/tt15398776/`).

Step 4: Critical Notes for Compliance

1. **Always check** `robots.txt` **first** before crawling a website.
2. **Respect** `Disallow` **rules** — avoid crawling prohibited pages (e.g., IMDB's search/list pages).
3. **No legal force, but high risk**: Even if `robots.txt` isn't legally binding, violating it may result in IP bans or legal action (especially for commercial use).

中华人民共和国刑法（285. 286. 287）

信息发布时间：2007-08-07　　字体：【 大 中 小】

第二百八十五条　违反国家规定，侵入国家事务、国防建设、尖端科学技术领域的计算机信息系统的，处三年以下有期徒刑或者拘役。

第二百八十六条　违反国家规定，对计算机信息系统功能进行删除、修改、增加、干扰，造成计算机信息系统不能正常运行，后果严重的，处五年以下有期徒刑或者拘役；后果特别严重的，处五年以上有期徒刑。

违反国家规定，对计算机信息系统中存储、处理或者传输的数据和应用程序进行删除、修改、增加的操作，后果严重的，依照前款的规定处罚。

故意制作、传播计算机病毒等破坏性程序，影响计算机系统正常运行，后果严重的，依照第一款的规定处罚。

第二百八十七条　利用计算机实施金融诈骗、盗窃、贪污、挪用公款、窃取国家秘密或者其他犯罪的，依照本法有关规定定罪处罚。

我国《刑法》第285-287条明确规定：

- 非法获取计算机信息系统数据罪
- 非法控制计算机信息系统罪
- 提供侵入、非法控制计算机信息系统程序、工具罪

《网络安全法》第27条：

> 任何个人和组织不得从事非法侵入他人网络、干扰他人网络正常功能、窃取网络数据等危害网络安全的活动

真实案例警示录

案例 1：某招聘网站爬虫案

某公司使用分布式爬虫每秒发起 500 次请求，导致目标网站瘫痪 3 小时。最终判决结果：

- 赔偿经济损失 80 万元
- 技术负责人被判有期徒刑 1 年（缓刑）

案例 2：电商价格监控案

某公司在爬取竞品价格数据时，绕过了网站的签名验证机制。法院认定其构成 “破坏计算机信息系统罪”，并处以 200 万元罚款

中文部分引自：CSDN 博主「kernelguru」原创文章，及济宁市人民政府官网。

原文链接（CSDN）：https://blog.csdn.net/kernelguru/article/details/148049096

（济宁市人民政府）：中华人民共和国刑法（285.286.287）

2.Developer Mode

Browser Developer Mode (also called Developer Tools) is a built-in set of debugging tools in modern browsers (Chrome, Firefox, Edge, etc.). It is the **core tool** for analyzing web page structure and locating target data when writing crawlers — it helps you quickly find the HTML tags/attributes where movie data (e.g., title, rating) is stored on IMDB pages.

Step 1: Open Developer Tools

There are 3 common ways to launch Developer Tools (take Chrome as an example):

1. **Keyboard shortcut** (most efficient): Press `F12` (Windows/Linux) or `Cmd + Opt + I` (Mac).
2. **Right-click menu**: Right-click on any area of the target web page → select `Inspect` (or "检查" in Chinese).
3. **Browser menu**: Click the three-dot icon in the top-right corner → `More tools` → `Developer tools`.

Step 2: Core Panels for Crawler Development

The Developer Tools interface has multiple panels; focus on these 3 for crawler work:

Panel	Key Functions for Crawling
Elements	View/Edit HTML structure of the page; locate tags for target data (e.g., movie title/rating).
Network	Monitor all HTTP requests/responses (e.g., check if IMDB pages use API to load data).
Source	Inspect/debug raw source code (HTML/JS/CSS) of the page; set breakpoints to analyze dynamic data loading logic; locate embedded JSON data (e.g., **NEXT_DATA** in IMDB pages) used for rendering content.

Step 3: Practical Use (Locate IMDB Movie Data)

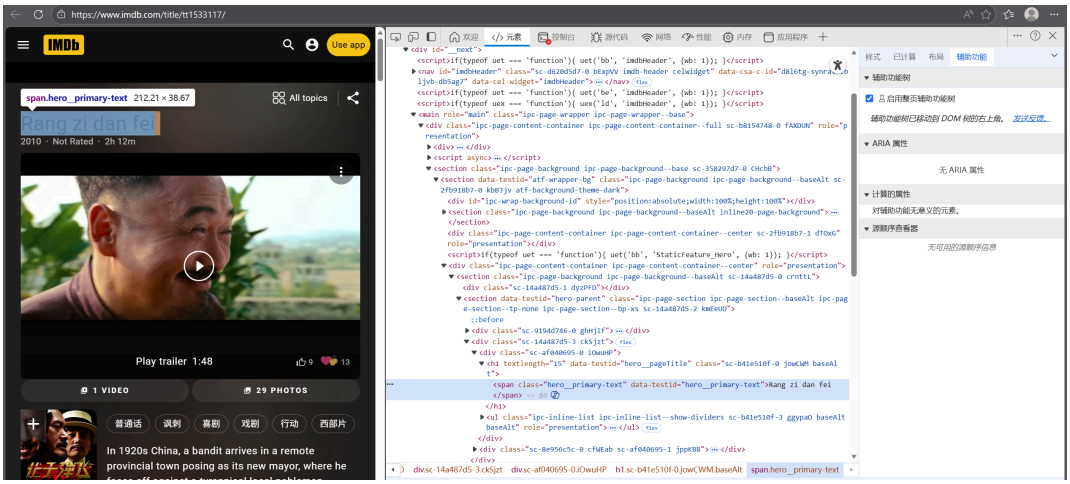
Let's use IMDB's movie detail page (e.g., `https://www.imdb.com/title/tt1533117/`) to demonstrate how to find data tags — this page features *Let the Bullets Fly* (my favorite movie, directed by Jiang Wen).

3.Locate Core Data Tags (Practical Demonstration)

We'll target key movie fields (title, director, rating, cast, release date, etc.) and find their corresponding HTML tags using the "Inspect Element" feature:

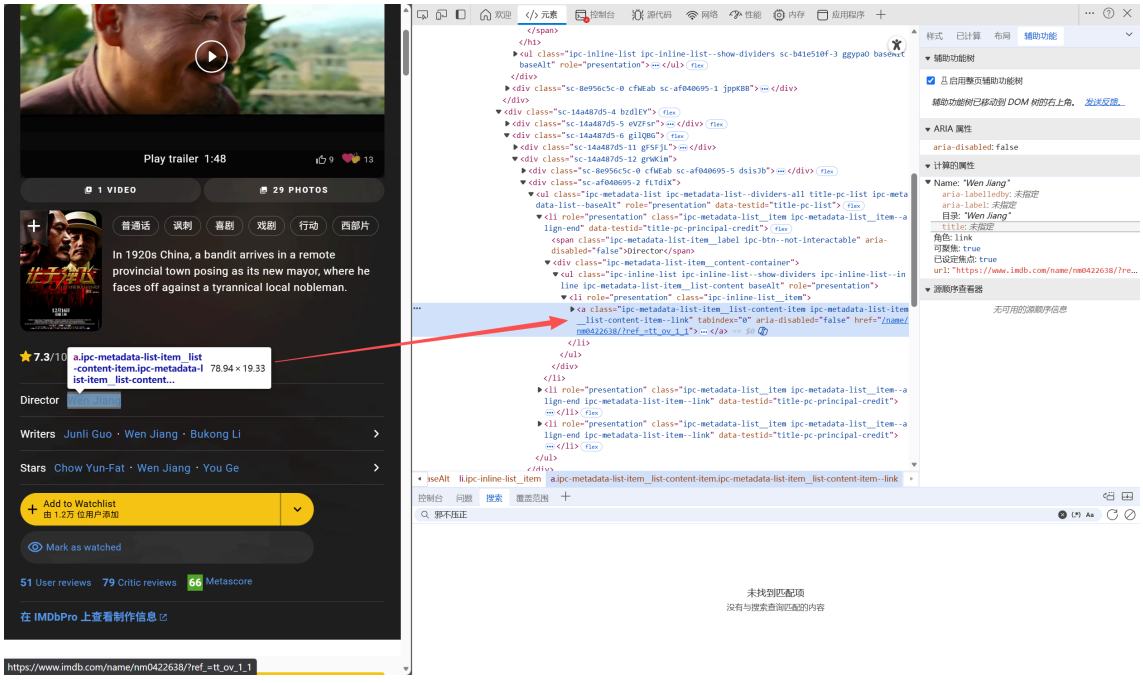
3.1 Locate Movie Title

- **Goal**: Find the tag for the title (*Rang zi dan fei*) and original title (*Rang zi dan fei*).
- **Operation**:
    1. Hover over the large title at the top of the page (e.g., "Rang zi dan fei").
    2. Right-click the title → Select **Inspect**.



3.2 Locate Director (Jiang Wen)

- **Goal**: Find the tag for the director "Wen Jiang" (Jiang Wen).
- **Operation**:
    1. Scroll down the page to the "Director" section.
    2. Hover over "Wen Jiang" → Right-click → Select **Inspect**.



3.3 Locate IMDB Rating (7.3/10)

> do it yourself!

3.4 Locate Key Cast (Chow Yun-Fat, You Ge, etc.)

> do it yourself

3.5 Locate Country, Year_released, run_time etc

> do it yourself

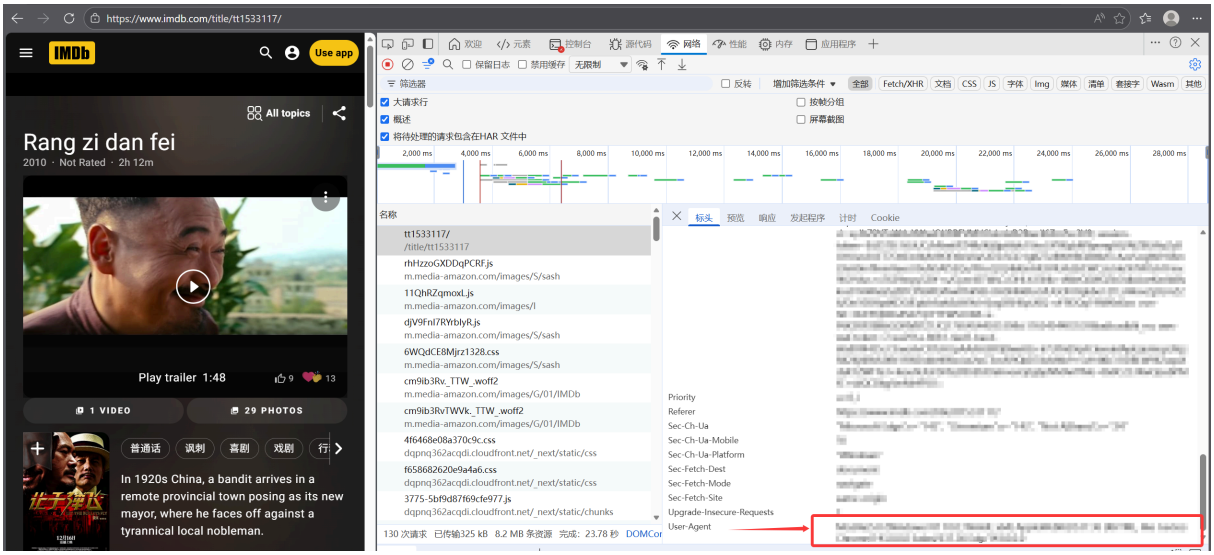
4.Python Code

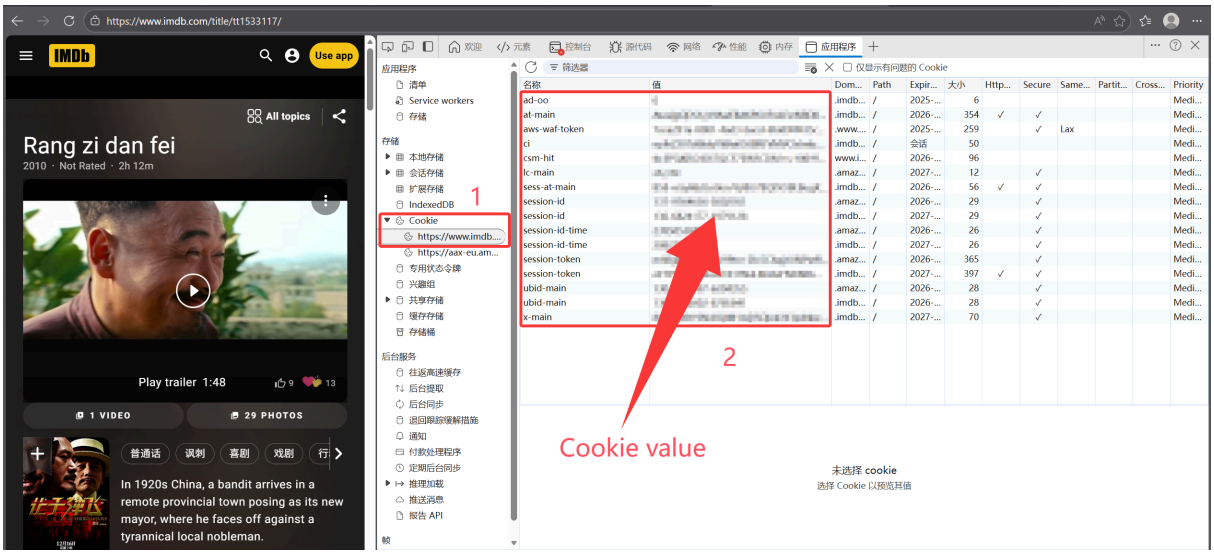
```
#import the necessary packets(use lxml here)
import json
import time
import random
import csv
import requests
from lxml import etree
import pandas as pd
```

```
def log(url):
    with open('database_record_t.txt', 'a') as f:
        f.write(url+'\n')

def get_full_url():

    df = pd.read_csv(
        'movies_tconst.csv',
        encoding='utf-8',
        usecols=['tconst'],  # 只读取 tconst 列（如 tt1234567）
        dtype={'tconst': str}  # 确保 tconst 是字符串类型（避免数字格式丢失前导 t）
    )

    total = len(df)
    print(f"开始处理 {total} 个影片链接...\n")

    for idx, row in df.iterrows():
        tconst = row['tconst']  # 获取当前行的 tconst（如 tt0120338）

        base_url = f"https://www.imdb.com/title/{tconst}/"  # 基础影片页面

        print(f"📌 第 {idx + 1}/{total} 个：{base_url}")

        # 访问影片页面（解析数据）
        get_movie_page(base_url)
        # 降低访问频率（防止影响网站正常运行）
        random_delay = random.uniform(3.5, 4)
        time.sleep(random_delay)
```

```
def get_movie_page(url):
    headers = {}
    cookie = {}
    """
```

```
    这里填写本机的header和该网站的cookie，依然在开发者工具处找（见下面图片）
    """
    with open('database_record_t.txt', 'r') as f:
        datas = f.readline()
    print('检查要下载文件是否已经在库中:')

    print("文件链接为url:", url)
    if url.strip('\n') in datas:
        print('该url已存在,检测下一个url...' + '\n')
    else:
        print('未检测到此条url对应的文件' + '\n' + '程序将继续往下执行')
        pass

        response = requests.get(url, headers=headers, cookies=cookie)
        sec = response.elapsed.seconds
        print(response.status_code, sec)
        html = response.text
        print('得到html')  # 检测程序运行情况
        get_info(html, url)
```





```
def get_info(html, url):
    """
    解析IMDB电影详情页的核心数据
    :param html: 网页HTML源码
    :param url: 目标网页URL（用于错误定位）
    :return: None（直接将数据写入CSV文件）
    """
    # 初始化XPath解析器
    content = etree.HTML(html)

    # 存储从JSON中解析的原始数据
    data = {}

    # ========== 第一步：提取并解析页面中的JSON数据 ==========
    try:
        # 定位__NEXT_DATA__脚本标签（IMDB核心数据存储位置）
        script_json = content.xpath('//script[@id="__NEXT_DATA__" and
@type="application/json"]/text()')[0]
        # 将JSON字符串转为Python字典
        data = json.loads(script_json)

    except IndexError:
        print(f"【错误】未找到__NEXT_DATA__标签，URL: {url}")
    except json.JSONDecodeError as e:
        print(f"【错误】JSON解析失败，URL: {url}，原因：{e}")
    except Exception as e:
        print(f"【错误】处理JSON时发生未知异常，URL: {url}，原因：{e}")

    # ========== 第二步：提取电影基础信息（兼容数据缺失场景） ==========
    # 提取电影唯一标识tconst（IMDB主键）
    try:
        tconst = data.get('props', {}).get('pageProps', {}).get('tconst')
    except Exception:  # 替换过窄的IndexError，兼容所有异常
        tconst = 'no info'

    # 提取核心数据容器（aboveTheFoldData包含电影主要信息）
    try:
        original_data = data.get('props', {}) \
            .get('pageProps', {}) \
            .get('aboveTheFoldData', {})
    except (AttributeError, TypeError):
        original_data = 'no info'

    # 提取原始片名（兼容非字典格式）
    try:
        original_title_text = original_data.get('originalTitleText')
        # 若为字典则取text字段，否则置空
        titles = original_title_text.get('text') if isinstance(original_title_text, dict)
else None
    except (AttributeError, TypeError):
        titles = 'no info'

    # 提取上映年份
```

```
    try:
        original_year = original_data.get('releaseYear', {})
        year = original_year.get('year') if isinstance(original_year, dict) else None
    except Exception:  # 替换过窄的IndexError，兼容所有异常
        year = 'no info'

    # 提取影片时长（秒转分钟，取整数）
    try:
        runsec = original_data.get('runtime', {})
        runtime = int(runsec.get('seconds')/60) if (isinstance(runsec, dict) and
runsec.get('seconds')) else None
    except Exception:  # 替换过窄的IndexError，兼容所有异常
        runtime = 'no info'

    # 提取IMDB评分
    try:
        original_rating = original_data.get('ratingsSummary', {})
        rating = original_rating.get('aggregateRating') if isinstance(original_rating, dict)
else None
    except Exception:  # 替换过窄的IndexError，兼容所有异常
        rating = 'no info'

    # 提取上映国家
    try:
        original_country = original_data.get('releaseDate', {})
        country_text = original_country.get('country', {}) if isinstance(original_country,
dict) else None
        country = country_text.get('text') if isinstance(country_text, dict) else None
    except Exception:  # 替换过窄的IndexError，兼容所有异常
        country = 'no info'

    # ========== 第三步：提取影人关联信息（导演/编剧/演员） ==========
    # 存储影人映射数据（tconst-IMDB_ID-角色类型）
    credit_ids = []
    try:
        # 逐层提取影人列表（每一层兜底为空，避免KeyError）
        principal_credits = data.get('props', {}) \
            .get('pageProps', {}) \
            .get('aboveTheFoldData', {}) \
            .get('principalCreditsV2', [])

        # 遍历影人分组（Director/Writers/Stars）
        for credit_group in principal_credits:
            # 跳过非字典项，防止后续取值报错
            if not isinstance(credit_group, dict):
                continue

            # 提取分组名称（如 Director/Directors/Writers/Stars）
            grouping = credit_group.get('grouping', {})
            grouping_text = grouping.get('text', '').strip()

            # 处理导演分组（D = Director）
            if grouping_text in ['Director', 'Directors']:
```

```
                credits = credit_group.get('credits', [])
                for credit in credits:
                    if not isinstance(credit, dict):
                        continue
                    # 提取导演唯一ID
                    name = credit.get('name', {})
                    person_id = name.get('id')
                    # 过滤空ID，保证数据有效性
                    if person_id and person_id.strip():
                        credit_ids.append({
                            'tconst': tconst,
                            'IMDB_ID': person_id,
                            'credited_as': 'D'  # D=导演
                        })

            # 处理编剧分组（W = Writer）
            elif grouping_text in ['Writer', 'Writers']:
                credits = credit_group.get('credits', [])
                for credit in credits:
                    if not isinstance(credit, dict):
                        continue
                    # 提取编剧唯一ID
                    name = credit.get('name', {})
                    person_id = name.get('id')
                    if person_id and person_id.strip():
                        credit_ids.append({
                            'tconst': tconst,
                            'IMDB_ID': person_id,
                            'credited_as': 'W'  # W=编剧
                        })

            # 处理演员分组（A = Actor）
            elif grouping_text in ['Star', 'Stars']:
                credits = credit_group.get('credits', [])
                for credit in credits:
                    if not isinstance(credit, dict):
                        continue
                    # 提取演员唯一ID
                    name = credit.get('name', {})
                    person_id = name.get('id')
                    if person_id and person_id.strip():
                        credit_ids.append({
                            'tconst': tconst,
                            'IMDB_ID': person_id,
                            'credited_as': 'A'  # A=演员
                        })

    except Exception as e:
        print(f"【错误】提取影人ID失败，URL: {url}，原因: {str(e)}")

    # ========== 第四步：写入CSV文件 ==========
    # 整理电影基础数据
    movie_data = {
```

```
            'tconst': tconst,           # IMDB电影唯一标识
            'titles': titles,           # 原始片名
            'country_fullname': country,  # 上映国家
            'year_released': year,      # 上映年份
            'runtime': runtime,         # 影片时长（分钟）
            'rating': rating            # IMDB评分
        }

        # 写入电影基础数据CSV（追加模式，UTF-8编码避免乱码）
        with open('movie_info_IMDB.csv', 'a', newline='', encoding='utf-8') as f:
            writer = csv.DictWriter(f, fieldnames=movie_data.keys())
            writer.writerow(movie_data)

        # 写入影人映射数据CSV（批量写入，效率更高）
        if credit_ids:  # 仅当有数据时写入，避免空行
            with open('person_mapping_IMDB.csv', 'a', newline='', encoding='utf-8') as f:
                writer = csv.DictWriter(f, fieldnames=['tconst','IMDB_ID','credited_as'])
                writer.writerows(credit_ids)

        # 清空列表，防止后续复用导致数据污染
        credit_ids.clear()

        # 记录爬取日志
        log(url)
        print(f"URL {url} 已处理完成，数据写入CSV\n" + '-' * 60 + '\n\n')
```

```
def main():
    get_full_url()


if __name__ == '__main__':
    main()
```

5.Exercise

This exercise focuses on **checking website crawling rules** (via `robots.txt`) and **locating target data in HTML source code** (using browser developer tools).

Task 1: Find & Analyze `robots.txt` for 2 Websites

Complete the following steps for **Douban Movie** (`https://movie.douban.com/`) and **Southern University of Science and Technology (SUSTech)** (`https://www.sustech.edu.cn/`):

Step 1: Access `robots.txt`

For any website, `robots.txt` is stored in the **root directory** (the main URL without subpaths).

- For Douban Movie: Visit `https://movie.douban.com/robots.txt`
- For SUSTech: Visit `https://www.sustech.edu.cn/robots.txt`

Step 2: Analysis `robots.txt`

Find the **actual core content** and analysis the `robots.txt` for Douban Movie. What is the actual core content of Douban Movie's (https://movie.douban.com/) `robots.txt` file, and specifically which paths are disallowed for crawlers?

Task 2: Locate Movie Data for *Hidden Man* in Douban Page Source Code

Target URL: `https://movie.douban.com/subject/26366496/` (Douban page for *邪不压正 / Hidden Man*)

Step 1: Open the Target Page & Developer Tools

1. Visit the URL in Chrome/Firefox/Edge.
2. Open **Developer Tools**:
    - Shortcut: `F12` (Windows/Linux) or `Cmd + Opt + I` (Mac)
    - Right-click any area on the page → select **Inspect** (检查)

Step 2: Locate Core Movie Data via Search & Inspect

Extract **6 key fields** (title, country, rating, release date, runtime, IMDb primary key) from the HTML code.

```
<script type="application/ld+json">
{
  "@context": "http://schema.org",
  "name": "邪不压正",
  "url": "/subject/26366496/",
  "image": "https://img2.doubanio.com/view/photo/s_ratio_poster/public/p2526297221.webp",
  .......
  "datePublished": "2018-07-13",
  "genre": ["\u5267\u60c5", "\u559c\u5267", "\u52a8\u4f5c"],
  "duration": "PT2H17M",
  "description": "七七事变前夕，华裔青年小亨德勒（彭于晏 饰）从美国远赴重洋，回到阔别十数年之久的北平从医。然而他真正的名字叫李天然，十三岁那年曾亲眼目睹师父一家遭师兄朱潜龙（廖凡 饰）和日本人根本一郎（泽田谦也 饰）...",
  "@type": "Movie",
  "aggregateRating": {
    "@type": "AggregateRating",
    "ratingCount": "736977",
    "bestRating": "10",
    "worstRating": "2",
    "ratingValue": "7.0"
  }
}
</script>
```

```
<div id="info">
            <span><span class='pl'>导演</span>: <span class='attrs'><a
href="https://www.douban.com/personage/27227726/" rel="v:directedBy">姜文</a></span></span>
<br/>
            <span><span class='pl'>编剧</span>: <span class='attrs'><a
href="https://www.douban.com/personage/27227726/">姜文</a> / <a
href="https://www.douban.com/personage/27540306/">何冀平</a> / <a
href="https://www.douban.com/personage/27570190/">李非</a> / <a
href="https://www.douban.com/personage/27554152/">孙悦</a> / <a
href="https://www.douban.com/personage/27614555/">张北海</a></span></span><br/>
            <span><span class='pl'>主演</span>: <span class='attrs'><a
href="https://www.douban.com/personage/27219486/" rel="v:starring">彭于晏</a> / <a
href="https://www.douban.com/personage/27213092/" rel="v:starring">廖凡</a> / <a
href="https://www.douban.com/personage/27227726/" rel="v:starring">姜文</a> / <a
href="https://www.douban.com/personage/27243602/" rel="v:starring">周韵</a> / <a
href="https://www.douban.com/personage/27210959/" rel="v:starring">许晴</a> / <a
href="https://www.douban.com/personage/27211222/" rel="v:starring">泽田谦也</a> / <a
href="https://www.douban.com/personage/30271958/" rel="v:starring">安地</a> / <a
href="https://www.douban.com/personage/27481544/" rel="v:starring">史航</a> / <a
href="https://www.douban.com/personage/27547819/" rel="v:starring">李梦</a> / <a
href="https://www.douban.com/personage/27244725/" rel="v:starring">丁嘉丽</a> / <a
href="https://www.douban.com/personage/30388126/" rel="v:starring">陈曦</a></span></span>
<br/>
            <span class="pl">类型:</span> <span property="v:genre">剧情</span> / <span
property="v:genre">喜剧</span> / <span property="v:genre">动作</span><br/>
            <span class="pl">制片国家/地区:</span> 中国大陆<br/>
            <span class="pl">语言:</span> 汉语普通话 / 英语 / 日语 / 法语<br/>
            <span class="pl">上映日期:</span> <span property="v:initialReleaseDate"
content="2018-07-13(中国大陆)">2018-07-13(中国大陆)</span><br/>
            <span class="pl">片长:</span> <span property="v:runtime" content="137">137分钟
</span> / 134分钟(香港)<br/>
            <span class="pl">又名:</span> 侠隐 / Hidden Man<br/>
            <span class="pl">IMDb:</span> tt8434380<br>

</div>
```

Tips

- If you can't find `robots.txt` for a website, it means there are **no explicit crawling restrictions** (but you still need to comply with laws and website terms).
- For Douban's movie page, if the HTML structure is complex, use the **search function** in the `Elements` panel and enter whatever you want to quickly locate the code.

Appendix .2 Lab Material Enhancement

lab material enhancement

Week 4

Q7: List all titles(in table movies) that contain the word "man".

POSIX Regular Expressions

The pattern matching operators for POSIX regular expressions are as follows:

~: Matches the regular expression, case-sensitive.

~*: Matches the regular expression, case-insensitive.

!~: Does not match the regular expression, case-sensitive.

!~*: Does not match the regular expression, case-insensitive.

so we can also write as

```
select * from movies
where title ~* '\mman\M'
```

Week 5

Q2:List the cast of the American "Treasure Island" 1950 film (first name, surname, and display "Director" instead of "D" and "Actor" instead of "A"

2. List the cast of the American(us) "Treasure Island"(title) 1950 film (first name, surname, and display "Director" instead of "D" and "Actor" instead of "A"

22 rows

	first_name	surname	credited_as
1	Lionel	Barrymore	Actor
2	Wallace	Beery	Actor
3	Nigel	Bruce	Actor
4	Jackie	Cooper	Actor
5	Victor	Fleming	Director
6	Lewis	Stone	Actor

The answer in the Question file mistakenly gives the cast of 1934 film "Treasure Island".

Week 7

Q8: List all year and "Events" (films released time, people births time, people deaths time) that occurred between 1930 and 1935 ,pushed into a subquery to add a sort key

```
SELECT year, event
 FROM (
 SELECT m.year_released AS year,
 m.title || ' (' || c.country_name || ') was released' AS event,
 m.title AS sort_key
 FROM movies m
 JOIN
 countries c ON c.country_code = m.country
 WHERE m.year_released BETWEEN 1930 AND 1935
 UNION ALL
 SELECT born,
 trim(coalesce(first_name, '') || ' ' || surname || ' was born'),
 surname AS sort_key
 FROM people
 WHERE born BETWEEN 1930 AND 1935
 UNION ALL
 SELECT died,
 trim(coalesce(first_name, '') || ' ' || surname || ' died'),
 surname AS sort_key
 FROM people
 WHERE died BETWEEN 1930 AND 1935
 )
 x
ORDER BY year,sort_key;
```

better to take the sort_key as the event as sort key, instead of surname.

week 10

In this lab, several bad triggers were evoked.

Experiment 4: before + each row + table

```
CREATE TRIGGER check_before
BEFORE INSERT OR UPDATE OR DELETE ON emp
FOR EACH ROW
EXECUTE FUNCTION process_emp_check();--test, and see result in emp_log
insert into emp values (017,'贺小山', 7000, 1);
UPDATE emp set salary=8000 where id=017;
delete from emp where id=017;
CREATE TRIGGER check_before_insert
BEFORE INSERT ON emp
```

```
FOR EACH ROW
EXECUTE FUNCTION process_emp_check();
CREATE TRIGGER check_before_update
BEFORE UPDATE ON emp
FOR EACH ROW
EXECUTE FUNCTION process_emp_check();
CREATE TRIGGER check_before_delete
BEFORE DELETE ON emp
FOR EACH ROW
EXECUTE FUNCTION process_emp_check();
```

The code above will result in bad triggers, and will influence the operations afterwards and cause trouble. For me, I deleted every trigger I created after an Experiment was done. It's better to give a note for that or some may get confused.